

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import os

# List contents of My Drive to locate the zip file
drive_path = '/content/drive/MyDrive'
if os.path.exists(drive_path):
    print(f"Contents of '{drive_path}':")
    for item in os.listdir(drive_path):
        print(item)
else:
    print(f"'{drive_path}' does not exist. Please ensure Google Drive is moun
```

```
Contents of '/content/drive/MyDrive':
LPU
Colab Notebooks
Classroom
archive
CICIDS 2017.zip
CIC-IDS 2017
Untitled folder
CICIDS2017
CICIDS2017_compressed.csv
label_encoder.save
scaler.save
```

```
import tensorflow as tf
print("GPU Available:", tf.config.list_physical_devices('GPU'))
```

GPU Available: []

```
import pandas as pd
import numpy as np
import zipfile, os
import shutil # Import shutil for rmtree

# Paths
zip_path = "/content/drive/MyDrive/CICIDS 2017.zip" # Corrected zip file name
extract_path = "/content/CICIDS2017"
output_csv = "/content/drive/MyDrive/CICIDS2017.csv"

# Ensure the extraction directory is clean before extraction
if os.path.exists(extract_path):
    print(f"Removing existing extraction directory: '{extract_path}'")
    shutil.rmtree(extract_path)

# Extract ZIP
if os.path.exists(zip_path):
    print(f"Extracting {zip_path} to {extract_path}...")
    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
```

```

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)
print("Extraction complete.")
else:
    print(f"Error: Zip file not found at '{zip_path}'. Please check the path and file name.")

# Verify extracted files
extracted_files = [f for f in os.listdir(extract_path) if f.endswith(".csv")]
if not extracted_files:
    print(f"Error: No CSV files found in '{extract_path}' after extraction. Directory may be empty or files may have been removed.")
else:
    print(f"Found {len(extracted_files)} CSV files in '{extract_path}'.")
    print(f"First 5 extracted CSV files: {extracted_files[:5]}")

# Read and merge CSV files
csv_files = [os.path.join(extract_path, f)
              for f in os.listdir(extract_path) if f.endswith(".csv")]

# Only proceed with concatenation if csv_files is not empty
if csv_files:
    df_list = [pd.read_csv(f) for f in csv_files]
    data = pd.concat(df_list, ignore_index=True)

    # Clean column names
    data.columns = data.columns.str.strip()

    # Remove NaN and Infinite values
    data.replace([np.inf, -np.inf], np.nan, inplace=True)
    data.dropna(inplace=True)

    # Save the full dataset (removed sampling step)
    data.to_csv(output_csv, index=False) # Saving full 'data' instead of 'reduced'

    print("Dataset saved!")
    print("Shape:", data.shape)
else:
    print("No CSV files found to concatenate. Data processing skipped.")

```

```

Removing existing extraction directory: '/content/CICIDS2017'
Extracting /content/drive/MyDrive/CICIDS 2017.zip to /content/CICIDS2017...
Extraction complete.
Error: No CSV files found in '/content/CICIDS2017' after extraction. Directory may be empty or files may have been removed.
No CSV files found to concatenate. Data processing skipped.

```

```

import pandas as pd

data = pd.read_csv("/content/drive/MyDrive/CICIDS2017_compressed.csv")
print("Dataset shape:", data.shape)
print("\nLabel distribution:\n", data['Label'].value_counts())

```

Dataset shape: (134603, 79)

Label distribution:

| Label    |       |
|----------|-------|
| BENIGN   | 50000 |
| PortScan | 15000 |

```
DoS Hulk                15000
DDoS                    15000
DoS GoldenEye           10293
FTP-Patator             7935
SSH-Patator             5897
DoS slowloris           5796
DoS Slowhttptest        5499
Bot                     1956
Web Attack  Brute Force  1507
Web Attack  XSS          652
Infiltration            36
Web Attack  Sql Injection 21
Heartbleed              11
Name: count, dtype: int64
```

```
data.head()
```

|   | Destination<br>Port | Flow<br>Duration | Total<br>Fwd<br>Packets | Total<br>Backward<br>Packets | Total<br>Length<br>of Fwd<br>Packets | Total<br>Length<br>of Bwd<br>Packets | Fwd<br>Packet<br>Length<br>Max | Fwd<br>Packet<br>Length<br>Min |
|---|---------------------|------------------|-------------------------|------------------------------|--------------------------------------|--------------------------------------|--------------------------------|--------------------------------|
| 0 | 80                  | 11997            | 3                       | 1                            | 18                                   | 0                                    | 6                              | 6                              |
| 1 | 9099                | 68               | 1                       | 1                            | 2                                    | 6                                    | 2                              | 2                              |
| 2 | 80                  | 55963607         | 2                       | 0                            | 0                                    | 0                                    | 0                              | 0                              |
| 3 | 21                  | 9870701          | 9                       | 15                           | 106                                  | 188                                  | 23                             | 0                              |
| 4 | 53458               | 32791            | 6                       | 1                            | 453                                  | 6                                    | 305                            | 6                              |

```
5 rows × 79 columns
```

```
#pd.read_csv(, encoding="ISO-8859-1")
data.info()
```

```

41 Packet Length Std      134603 non-null float64
42 Packet Length Variance 134603 non-null float64
43 FIN Flag Count         134603 non-null int64
44 SYN Flag Count         134603 non-null int64
45 RST Flag Count         134603 non-null int64
46 PSH Flag Count         134603 non-null int64
47 ACK Flag Count         134603 non-null int64
48 URG Flag Count         134603 non-null int64
49 CWE Flag Count         134603 non-null int64
50 ECE Flag Count         134603 non-null int64
51 Down/Up Ratio          134603 non-null int64
52 Average Packet Size    134603 non-null float64
53 Avg Fwd Segment Size   134603 non-null float64
54 Avg Bwd Segment Size   134603 non-null float64
55 Fwd Header Length.1    134603 non-null int64
56 Fwd Avg Bytes/Bulk     134603 non-null int64
57 Fwd Avg Packets/Bulk   134603 non-null int64
58 Fwd Avg Bulk Rate       134603 non-null int64
59 Bwd Avg Bytes/Bulk     134603 non-null int64
60 Bwd Avg Packets/Bulk   134603 non-null int64
61 Bwd Avg Bulk Rate       134603 non-null int64
62 Subflow Fwd Packets    134603 non-null int64
63 Subflow Fwd Bytes      134603 non-null int64
64 Subflow Bwd Packets    134603 non-null int64
65 Subflow Bwd Bytes      134603 non-null int64
66 Init_Win_bytes_forward 134603 non-null int64
67 Init_Win_bytes_backward 134603 non-null int64
68 act_data_pkt_fwd       134603 non-null int64
69 min_seg_size_forward   134603 non-null int64
70 Active Mean            134603 non-null float64
71 Active Std             134603 non-null float64
72 Active Max             134603 non-null int64
73 Active Min             134603 non-null int64
74 Idle Mean              134603 non-null float64
75 Idle Std               134603 non-null float64
76 Idle Max               134603 non-null int64
77 Idle Min               134603 non-null int64
78 Label                  134603 non-null object

```

dtypes: float64(24), int64(54), object(1)

memory usage: 81.1+ MB

```

# Strip column names
data.columns = data.columns.str.strip()

print("Missing values:", data.isnull().sum().sum())

```

Missing values: 0

```
data.isnull().sum()
```

|                                    | 0   |
|------------------------------------|-----|
| <b>Destination Port</b>            | 0   |
| <b>Flow Duration</b>               | 0   |
| <b>Total Fwd Packets</b>           | 0   |
| <b>Total Backward Packets</b>      | 0   |
| <b>Total Length of Fwd Packets</b> | 0   |
| ...                                | ... |
| <b>Idle Mean</b>                   | 0   |
| <b>Idle Std</b>                    | 0   |
| <b>Idle Max</b>                    | 0   |
| <b>Idle Min</b>                    | 0   |
| <b>Label</b>                       | 0   |

79 rows × 1 columns

**dtype:** int64

```
data.isnull().sum()
```

|                                    | 0   |
|------------------------------------|-----|
| <b>Destination Port</b>            | 0   |
| <b>Flow Duration</b>               | 0   |
| <b>Total Fwd Packets</b>           | 0   |
| <b>Total Backward Packets</b>      | 0   |
| <b>Total Length of Fwd Packets</b> | 0   |
| ...                                | ... |
| <b>Idle Mean</b>                   | 0   |
| <b>Idle Std</b>                    | 0   |
| <b>Idle Max</b>                    | 0   |
| <b>Idle Min</b>                    | 0   |
| <b>Label</b>                       | 0   |

79 rows × 1 columns

**dtype:** int64

```
X = data.drop(columns=['Label'])
y = data['Label']
```

```
print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (134603, 78)
y shape: (134603,)
```

```
y = y.str.replace("❖", "-", regex=False)
```

```
from sklearn.preprocessing import LabelEncoder
```

```
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)
```

```
print("Classes:", label_encoder.classes_)
print("Encoded labels example:", y_encoded[:10])
```

```
Classes: ['BENIGN' 'Bot' 'DDoS' 'DoS GoldenEye' 'DoS Hulk' 'DoS Slowhttpstest'
'DoS slowloris' 'FTP-Patator' 'Heartbleed' 'Infiltration' 'PortScan'
'SSH-Patator' 'Web Attack - Brute Force' 'Web Attack - Sql Injection'
'Web Attack - XSS']
Encoded labels example: [ 4 10  3  7  0 10  4  0  0  0]
```

```
from sklearn.preprocessing import StandardScaler
import numpy as np
```

```
# Convert to numpy
X_values = X.values
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_values)
```

```
print("Scaled feature shape:", X_scaled.shape)
print("Sample scaled values:\n", X_scaled[:2])
```

```
Scaled feature shape: (134603, 78)
Sample scaled values:
[[-3.33776117e-01 -5.51279841e-01 -4.96603806e-02 -5.96613841e-02
 -3.61119712e-02 -3.19428808e-02 -2.89610903e-01 -8.35105406e-02
 -2.49679155e-01 -2.84730095e-01 -5.35730946e-01 -3.75279230e-01
 -5.67956594e-01 -5.06403649e-01 -5.03009335e-02 -2.58287675e-01
 -3.21366684e-01 -4.39081724e-01 -4.90014670e-01 -1.23126366e-01
 -5.33574078e-01 -3.32081275e-01 -4.23152344e-01 -4.84483556e-01
 -1.78239515e-01 -3.65421782e-01 -2.43891568e-01 -2.60756102e-01
 -3.08878868e-01 -1.50758267e-01 -2.74223085e-01  0.00000000e+00
 -6.09488757e-03  0.00000000e+00  2.89011344e-03  2.88664148e-03
 -2.35142845e-01 -1.92639305e-01 -3.51060613e-01 -5.48708366e-01
 -5.99753217e-01 -5.49904280e-01 -4.09188580e-01  5.20810147e+00
 -2.74223085e-01 -1.21904544e-02 -9.13552497e-01 -7.28140195e-01
 -2.89503752e-01 -6.09488757e-03 -1.21904544e-02 -9.49467296e-01
 -6.00056171e-01 -2.49679155e-01 -5.67956594e-01  2.89011344e-03
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00]
```

```

0.00000000e+00 0.00000000e+00 -4.96603806e-02 -3.61119712e-02
-5.96613841e-02 -3.19402464e-02 -7.18975378e-01 -1.81768721e-01
-1.88420529e-02 3.10838050e-03 -2.26421997e-01 -1.34067079e-01
-2.34380057e-01 -1.88593840e-01 -4.40815409e-01 -1.91971137e-01
-4.63365960e-01 -4.12602230e-01]
[ 3.17083737e-01 -5.51613203e-01 -8.07102722e-02 -5.96613841e-02
-3.71509120e-02 -3.19093031e-02 -2.96630046e-01 -1.26429352e-01
-2.73294305e-01 -2.84730095e-01 -5.33543154e-01 -2.61594278e-01
-5.60740286e-01 -5.06403649e-01 -4.65758275e-02 -1.32484765e-01
-3.21785573e-01 -4.39713871e-01 -4.90404985e-01 -1.23117916e-01
-5.33908449e-01 -3.32480959e-01 -4.23867760e-01 -4.84877264e-01
-1.78239584e-01 -3.65421782e-01 -2.43891568e-01 -2.60756102e-01
-3.08878868e-01 -1.50758267e-01 -2.74223085e-01 0.00000000e+00
-6.09488757e-03 0.00000000e+00 2.87785005e-03 2.88255368e-03
-1.70346633e-01 1.93967781e-01 -2.70628867e-01 -5.48708366e-01
-6.03503513e-01 -5.50334095e-01 -4.09189354e-01 -1.92008548e-01
-2.74223085e-01 -1.21904544e-02 1.09462784e+00 -7.28140195e-01
-2.89503752e-01 -6.09488757e-03 -1.21904544e-02 6.41920193e-01
-6.02368948e-01 -2.73294305e-01 -5.60740286e-01 2.87785005e-03
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 -8.07102722e-02 -3.71509120e-02
-5.96613841e-02 -3.19066702e-02 -6.47768779e-01 -2.16242870e-01
-5.48463523e-02 3.11108123e-03 -2.26421997e-01 -1.34067079e-01
-2.34380057e-01 -1.88593840e-01 -4.40815409e-01 -1.91971137e-01
-4.63365960e-01 -4.12602230e-01]]

```

```

import joblib
joblib.dump(scaler, "/content/drive/MyDrive/scaler.save") # scaled values sav

['/content/drive/MyDrive/scaler.save']

```

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)

print("Training set shape:", X_train.shape)
print("Testing set shape:", X_test.shape)

```

```

Training set shape: (107682, 78)
Testing set shape: (26921, 78)

```

```

# Reshape for CNN
X_train_cnn = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
X_test_cnn = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)

print("CNN Train shape:", X_train_cnn.shape)
print("CNN Test shape:", X_test_cnn.shape)

```

```
CNN Train shape: (107682, 78, 1)
CNN Test shape: (26921, 78, 1)
```

```
# Updated Model
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatten, Dense, Dropout
from tensorflow.keras.optimizers import Adam

num_classes = len(label_encoder.classes_)

model = Sequential([
    Conv1D(filters=64, kernel_size=3, activation='relu',
           input_shape=(X_train_cnn.shape[1], 1)),
    MaxPooling1D(pool_size=2),
    Conv1D(filters=128, kernel_size=3, activation='relu'),
    MaxPooling1D(pool_size=2),

    Flatten(),

    Dense(128, activation='relu'),
    Dense(128, activation='relu'),
    Dropout(0.3),

    Dense(num_classes, activation='softmax')
])

model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()
```



```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv2d.py:10:
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

| Layer (type)                   | Output Shape    | Param # |
|--------------------------------|-----------------|---------|
| conv1d (Conv1D)                | (None, 76, 64)  | 256     |
| max_pooling1d (MaxPooling1D)   | (None, 38, 64)  | 0       |
| conv1d_1 (Conv1D)              | (None, 36, 128) | 24,704  |
| max_pooling1d_1 (MaxPooling1D) | (None, 18, 128) | 0       |
| flatten (Flatten)              | (None, 2304)    | 0       |
| dense (Dense)                  | (None, 128)     | 295,040 |
| dense_1 (Dense)                | (None, 128)     | 16,512  |
| dropout (Dropout)              | (None, 128)     | 0       |
| dense_2 (Dense)                | (None, 15)      | 1,935   |

Total params: 338,447 (1.29 MB)

Trainable params: 338,447 (1.29 MB)

Non-trainable params: 0 (0.00 MB)

```
# Updated Training
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=35,
    restore_best_weights=True
)

reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=10,
    min_lr=1e-5
)

history = model.fit(
    X_train_cnn,
    y_train,
    epochs=30,
    batch_size=256,
    validation_split=0.2,
    callbacks=[early_stop, reduce_lr],
    verbose=1
)
```

Epoch 1/30

337/337 ————— 29s 80ms/step - accuracy: 0.8690 - loss: 0.4318

Epoch 2/30

```

337/337 ----- 40s 77ms/step - accuracy: 0.9535 - loss: 0.1533
Epoch 3/30
337/337 ----- 41s 78ms/step - accuracy: 0.9610 - loss: 0.1221
Epoch 4/30
337/337 ----- 26s 76ms/step - accuracy: 0.9647 - loss: 0.1069
Epoch 5/30
337/337 ----- 44s 86ms/step - accuracy: 0.9673 - loss: 0.0945
Epoch 6/30
337/337 ----- 26s 78ms/step - accuracy: 0.9696 - loss: 0.0870
Epoch 7/30
337/337 ----- 27s 79ms/step - accuracy: 0.9704 - loss: 0.0825
Epoch 8/30
337/337 ----- 24s 71ms/step - accuracy: 0.9723 - loss: 0.0766
Epoch 9/30
337/337 ----- 27s 79ms/step - accuracy: 0.9758 - loss: 0.0671
Epoch 10/30
337/337 ----- 30s 90ms/step - accuracy: 0.9775 - loss: 0.0626
Epoch 11/30
337/337 ----- 26s 77ms/step - accuracy: 0.9772 - loss: 0.0617
Epoch 12/30
337/337 ----- 40s 74ms/step - accuracy: 0.9786 - loss: 0.0583
Epoch 13/30
337/337 ----- 26s 78ms/step - accuracy: 0.9791 - loss: 0.0581
Epoch 14/30
337/337 ----- 27s 80ms/step - accuracy: 0.9790 - loss: 0.0620
Epoch 15/30
337/337 ----- 40s 77ms/step - accuracy: 0.9797 - loss: 0.0551
Epoch 16/30
337/337 ----- 42s 80ms/step - accuracy: 0.9803 - loss: 0.0537
Epoch 17/30
337/337 ----- 39s 75ms/step - accuracy: 0.9800 - loss: 0.0534
Epoch 18/30
337/337 ----- 41s 74ms/step - accuracy: 0.9808 - loss: 0.0517
Epoch 19/30
337/337 ----- 41s 74ms/step - accuracy: 0.9811 - loss: 0.0509
Epoch 20/30
337/337 ----- 40s 72ms/step - accuracy: 0.9813 - loss: 0.0507
Epoch 21/30
337/337 ----- 42s 75ms/step - accuracy: 0.9815 - loss: 0.0496
Epoch 22/30
337/337 ----- 40s 73ms/step - accuracy: 0.9810 - loss: 0.0500
Epoch 23/30
337/337 ----- 41s 75ms/step - accuracy: 0.9815 - loss: 0.0496
Epoch 24/30
337/337 ----- 41s 74ms/step - accuracy: 0.9821 - loss: 0.0480
Epoch 25/30
337/337 ----- 40s 72ms/step - accuracy: 0.9827 - loss: 0.0458
Epoch 26/30
337/337 ----- 41s 73ms/step - accuracy: 0.9824 - loss: 0.0472
Epoch 27/30
337/337 ----- 27s 81ms/step - accuracy: 0.9824 - loss: 0.0467
Epoch 28/30
337/337 ----- 26s 77ms/step - accuracy: 0.9829 - loss: 0.0471
Epoch 29/30
-- --

```

```
test_loss, test_accuracy = model.evaluate(X_test_cnn, y_test, verbose=0)
```

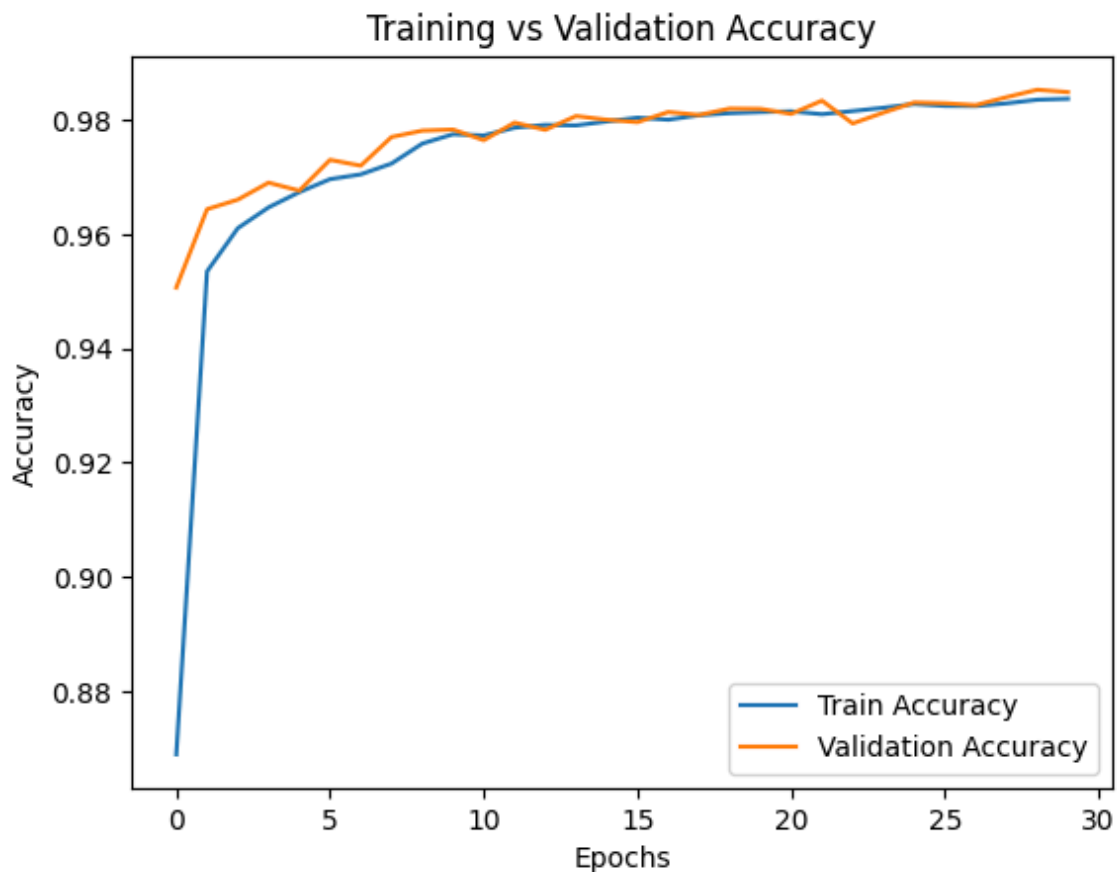
```
print(f"Test Accuracy: {test_accuracy*100:.2f}%")
```

```
print(f"Test Loss: {test_loss:.4f}")
```

Test Accuracy: 98.38%

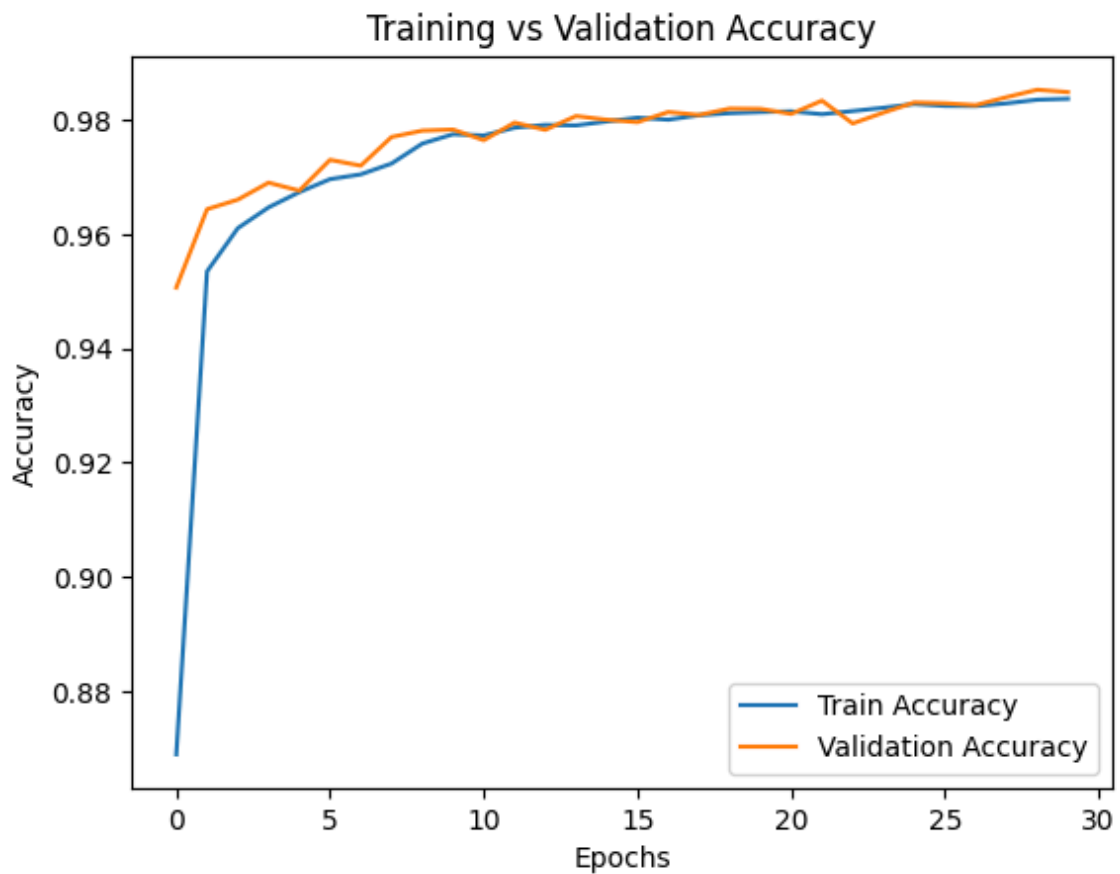
Test Loss: 0.0461

```
# 99.72%
import matplotlib.pyplot as plt
plt.figure()
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Training vs Validation Accuracy')
plt.legend()
plt.show()
```



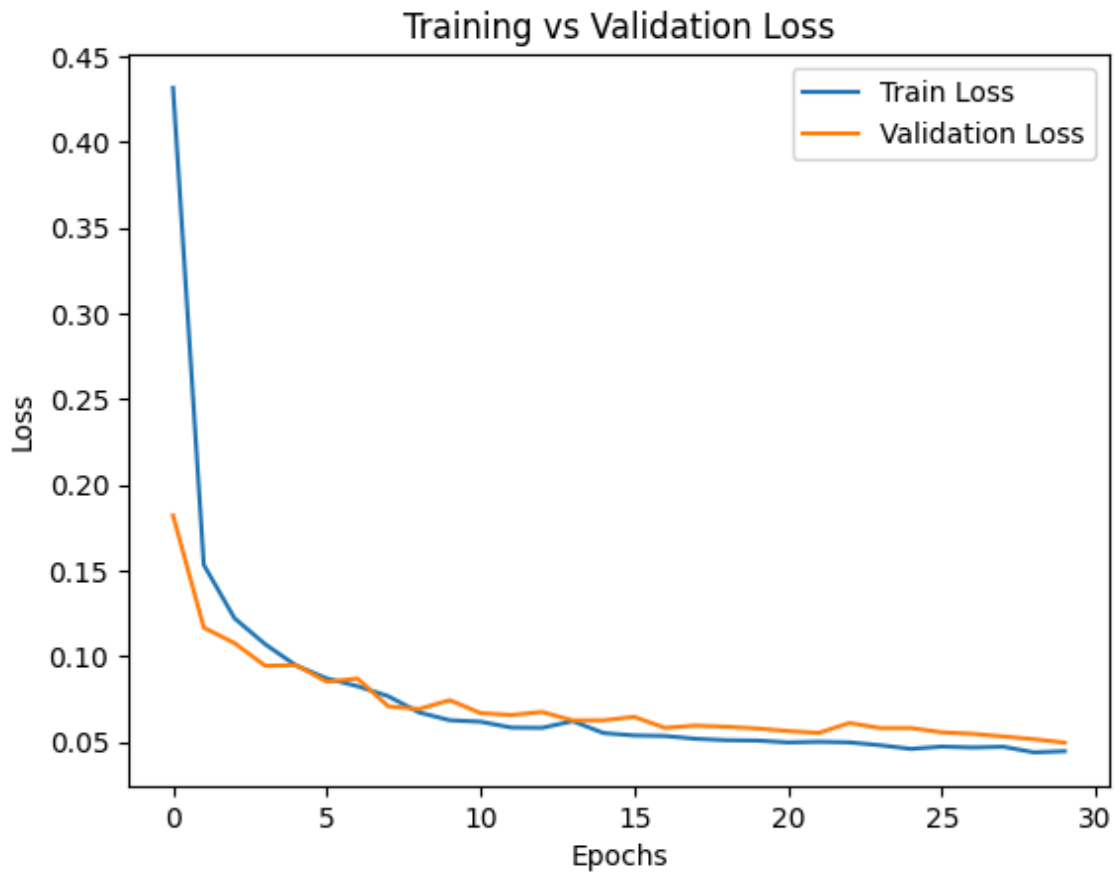
```
# 99.72%
import matplotlib.pyplot as plt

plt.figure()
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Training vs Validation Accuracy')
plt.legend()
plt.show()
```



```
# 99.72%
import matplotlib.pyplot as plt

plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training vs Validation Loss')
plt.legend()
plt.show()
```



```
# 99.72%
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix
import numpy as np

y_pred = model.predict(X_test_cnn)
y_pred_classes = np.argmax(y_pred, axis=1)

cm = confusion_matrix(y_test, y_pred_classes)

plt.figure(figsize=(14, 12))
sns.heatmap(cm,
            annot=True,
            fmt='d',
            cmap='Blues',
            xticklabels=label_encoder.classes_,
            yticklabels=label_encoder.classes_)
plt.title('Confusion Matrix - CNN IDS')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.xticks(rotation=90)
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
```

842/842

4s 4ms/step



# 99.72%

import matplotlib.pyplot as plt

import pandas as pd

# Get true label distribution

true\_label\_counts = pd.Series(y\_test).map(lambda x: label\_encoder.classes\_[x]).

# Get predicted label distribution

predicted\_label\_counts = pd.Series(y\_pred\_classes).map(lambda x: label\_encoder.

# Combine for easier plotting (aligning indices)

```
combined_counts = pd.DataFrame({
    'True Labels': true_label_counts,
    'Predicted Labels': predicted_label_counts
}).fillna(0)
```

# Plotting

plt.figure(figsize=(15, 7))

combined\_counts.plot(kind='bar', figsize=(15, 7))

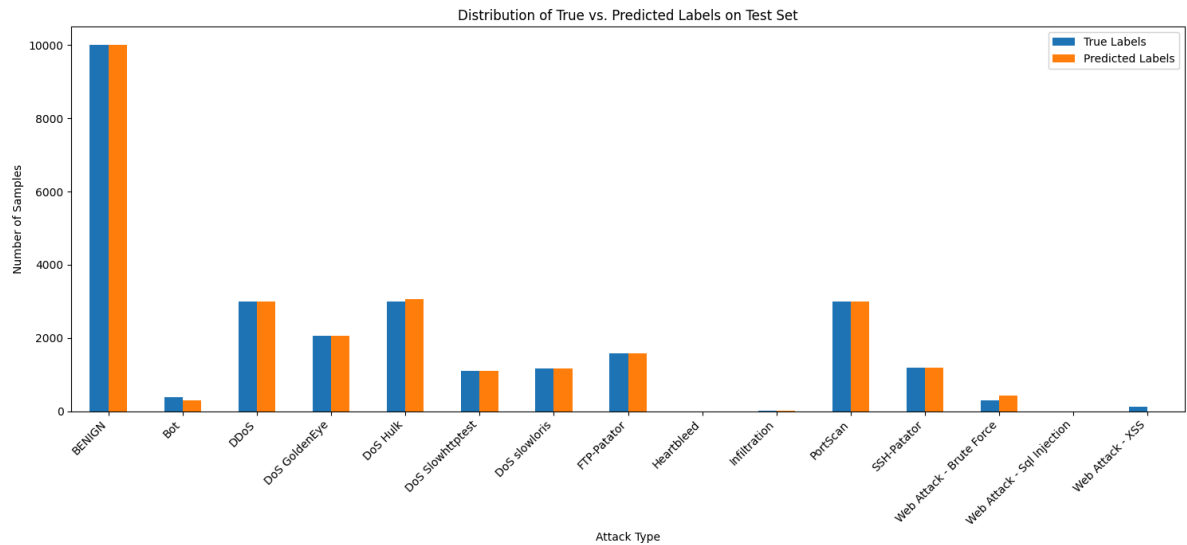
plt.title('Distribution of True vs. Predicted Labels on Test Set')

plt.xlabel('Attack Type')

plt.ylabel('Number of Samples')

```
plt.xticks(rotation=45, ha='right')
plt.legend()
plt.tight_layout()
plt.show()
```

<Figure size 1500x700 with 0 Axes>



## ✓ Saving Files

```
import tensorflow as tf

# Save the trained CNN model in native Keras format
model.save("/content/drive/MyDrive/IDS_model.keras")
print("Saved to Google Drive.")
```

Saved to Google Drive.

```
import joblib

# Save the label encoder
joblib.dump(label_encoder, "/content/drive/MyDrive/label_encoder_final.save")
print("Label Encoder saved to Google Drive.")
```

Label Encoder saved to Google Drive.

Start coding or [generate](#) with AI.

## END OF CODE

## ✓ Deployment

```
# Streamlit installation
import sys
!{sys.executable} -m pip install streamlit

print("Streamlit installed.")
```

[Show hidden output](#)

```
%%writefile app.py
import streamlit as st
import tensorflow as tf
import joblib
import numpy as np
import pandas as pd

# Load the saved CNN model
loaded_model = tf.keras.models.load_model('/content/drive/MyDrive/IDS_model.k

# Load the scaler
loaded_scaler = joblib.load('/content/drive/MyDrive/scaler.save')

# Load the label encoder
loaded_label_encoder = joblib.load('/content/drive/MyDrive/label_encoder_full

# Streamlit App
st.title('Network Intrusion Detection System (NIDS) CNN Model')
st.write('Upload a CSV file containing network traffic features for intrusion

uploaded_file = st.file_uploader('Choose a CSV file', type='csv')

if uploaded_file is not None:
    try:
        # Read the uploaded CSV file
        input_df = pd.read_csv(uploaded_file)
        st.write('Uploaded Data Preview:')
        st.write(input_df.head())

        # Preprocess the input data
        # Ensure columns are in the same order as the training data (assuming
        # For simplicity, we'll assume the uploaded CSV has the same columns
        # In a real application, you'd need more robust column handling (e.g.

        # Scale the features
        input_scaled = loaded_scaler.transform(input_df.values)

        # Reshape for CNN model (add channel dimension)
        input_cnn = input_scaled.reshape(input_scaled.shape[0], input_scaled.

        # Make predictions
        predictions = loaded_model.predict(input_cnn)
        predicted_classes_encoded = np.argmax(predictions, axis=1)
        predicted_labels = loaded_label_encoder.inverse_transform(predicted_c

        st.write('Prediction Results:')
        result_df = input_df.copy()
```



```

result_df['Predicted_Attack_Type'] = predicted_labels
st.write(result_df[['Flow Duration', 'Destination Port', 'Predicted_A

# Display value counts for predicted labels
st.write('Predicted Attack Type Distribution:')
st.write(pd.Series(predicted_labels).value_counts())

except Exception as e:
    st.error(f'Error processing file: {e}')

```

Overwriting app.py

First, we will start the Streamlit application. Instead of `streamlit run app.py`, which sometimes encounters path issues in Colab, we'll use `python -m streamlit run app.py`.

```

import subprocess
import sys

# Run the Streamlit app in the background
# Using 'python -m streamlit' explicitly calls the module
# The --server.port argument is crucial for ngrok to tunnel to the correct po
process = subprocess.Popen([sys.executable, '-m', 'streamlit', 'run', 'app.py

print("Streamlit app is running in the background on port 8501.")
print("Next, we'll set up ngrok to create a public URL for this app.")

```

Streamlit app is running in the background on port 8501.  
Next, we'll set up ngrok to create a public URL for this app.

Now, to make your Streamlit app accessible via a public URL, you'll need to set up `ngrok`. If you haven't already, you'll need an ngrok authentication token.

1. **Visit ngrok website:** Go to [ngrok.com](https://ngrok.com) and sign up for a free account if you don't have one.
2. **Obtain Auth Token:** Log in to your ngrok dashboard and navigate to the 'Your Authtoken' section (usually found under 'Setup & Installation'). Copy your authentication token.
3. **Set Auth Token:** Execute the following cell, replacing `<YOUR_NGROK_AUTH_TOKEN>` with your actual token.

```

from pyngrok import ngrok

# Replace <YOUR_NGROK_AUTH_TOKEN> with your actual token
ngrok.set_auth_token("YOUR_NGROK_AUTH_TOKEN_HERE") # IMPORTANT: Replace this

print("ngrok authentication token set.")

```

```
from pyngrok import ngrok

# Open a ngrok tunnel to the Streamlit app's port (8501)
public_url = ngrok.connect(addr='8501', proto='http')

print(f"Public URL: {public_url}")
print("You can access your Streamlit app via this URL. Keep this Colab tab op
```

```
import subprocess
import sys

# Run the Streamlit app in the background
# The --server.port argument is crucial for ngrok to tunnel to the correct po
process = subprocess.Popen(['streamlit', '-m', 'run', 'app.py', '--server.port

print("Streamlit app is running. You might need to expose the port using ngro
print("To stop the app, interrupt this kernel or kill the process manually.")
```

[Show hidden output](#)

```
import sys
!{sys.executable} -m pip install pyngrok

print("pyngrok installed.")
```

```
Collecting pyngrok
  Downloading pyngrok-8.0.0-py3-none-any.whl.metadata (8.2 kB)
Requirement already satisfied: PyYAML>=5.1 in /usr/local/lib/python3.12/dist-p
Downloading pyngrok-8.0.0-py3-none-any.whl (24 kB)
Installing collected packages: pyngrok
Successfully installed pyngrok-8.0.0
pyngrok installed.
```

```
from pyngrok import ngrok

# Replace <YOUR_NGROK_AUTH_TOKEN> with your actual token
ngrok.set_auth_token("3BdK21YaWdL8BHIjJsTopEuv2re_6Kmnwk9s6RQ6fR8bRH48p")

print("ngrok authentication token set.")

ngrok authentication token set.
```

Creating an ngrok tunnel to expose the Streamlit application running on port 8501 to a public URL.

```
ngrok.kill()
```

```
from pyngrok import ngrok
```

```
# Open a ngrok tunnel to the Streamlit app's port
public_url = ngrok.connect(addr='8501', proto='http')

print(f"Public URL: {public_url}")
print("You can access your Streamlit app via this URL.")
```

Public URL: NgrokTunnel: "<https://preinstructional-dannie-ungravely.ngrok-free>  
You can access your Streamlit app via this URL.

```
import pandas as pd
import os

# Reload data if not already in memory (or ensure 'data' is the full dataset)
# Assuming 'data' DataFrame from previous cells is available and correct

# Define target counts for diversity
sample_counts = {
    'BENIGN': 50000,
    'DoS Hulk': 10000,
    'PortScan': 10000,
    'DDoS': 10000,
    'DoS GoldenEye': 10000,
    'FTP-Patator': 7000,
    'SSH-Patator': 5000,
    'DoS slowloris': 5000,
    'DoS Slowhttptest': 5000,
    'Bot': 1500,
    'Web Attack - Brute Force': 1000,
    'Web Attack - XSS': 500,
    'Infiltration': 30,
    'Web Attack - Sql Injection': 20,
    'Heartbleed': 10
}

sampled_dfs = []

# Sample each class
for label, count in sample_counts.items():
    class_data = data[data['Label'] == label]
    # If the class has fewer samples than requested, take all of them
    n_samples = min(count, len(class_data))
    if n_samples > 0:
        sampled_dfs.append(class_data.sample(n=n_samples, random_state=42))
    else:
        print(f"Warning: Class '{label}' has no samples available in the data")

# Concatenate all sampled DataFrames
diverse_sampled_df = pd.concat(sampled_dfs, ignore_index=True)

# Drop the 'Label' column as it is the target for prediction in the app
X_diverse_for_app = diverse_sampled_df.drop(columns=['Label'])

# Define the path for the new diverse sampled CSV file
diverse_sampled_csv_path = '/content/drive/MyDrive/CICIDS2017_diverse_sample_
```

```
# Save the diverse DataFrame to CSV
X_diverse_for_app.to_csv(diverse_sampled_csv_path, index=False)

print(f"Diverse sampled CSV file created at: {diverse_sampled_csv_path}")
print(f"Shape of the diverse sampled data: {X_diverse_for_app.shape}")

# Verify the size of the new file
if os.path.exists(diverse_sampled_csv_path):
    file_size_bytes = os.path.getsize(diverse_sampled_csv_path)
    file_size_mb = file_size_bytes / (1024 * 1024)
    print(f"Size of the diverse sampled file: {file_size_mb:.2f} MB")
else:
    print(f"Error: Diverse sampled file not found at '{diverse_sampled_csv_pa
```

```
Warning: Class 'Web Attack - Brute Force' has no samples available in the data
Warning: Class 'Web Attack - XSS' has no samples available in the dataset.
Warning: Class 'Web Attack - Sql Injection' has no samples available in the da
Diverse sampled CSV file created at: /content/drive/MyDrive/CICIDS2017_diverse
Shape of the diverse sampled data: (113540, 78)
Size of the diverse sampled file: 37.11 MB
```

## ✓ Create a sampled CSV file without the 'Label' column for Streamlit app

```
import pandas as pd
import os

# Drop the 'Label' column as it is the target for prediction
X_for_app = data.drop(columns=['Label'])

# Take a smaller sample, e.g., the first 100,000 rows
sampled_df_for_app = X_for_app.head(100000)

# Define the path for the new sampled CSV file
sampled_csv_path = '/content/drive/MyDrive/CICIDS2017_sample_for_app.csv'

# Save the sampled DataFrame to CSV
sampled_df_for_app.to_csv(sampled_csv_path, index=False)

print(f"Sampled CSV file created at: {sampled_csv_path}")
print(f"Shape of the sampled data: {sampled_df_for_app.shape}")

# Verify the size of the new file
if os.path.exists(sampled_csv_path):
    file_size_bytes = os.path.getsize(sampled_csv_path)
    file_size_mb = file_size_bytes / (1024 * 1024)
    print(f"Size of the sampled file: {file_size_mb:.2f} MB")
else:
    print(f"Error: Sampled file not found at '{sampled_csv_path}'.")
```

Sampled CSV file created at: /content/drive/MyDrive/CICIDS2017\_sample\_for\_app.  
 Shape of the sampled data: (100000, 78)  
 Size of the sampled file: 31.06 MB

```
import pandas as pd
import tensorflow as tf
import joblib
import numpy as np

# Load the main dataset
data = pd.read_csv('/content/drive/MyDrive/CICIDS2017.csv')
print("Main dataset loaded. Shape:", data.shape)

# Load the trained CNN model
loaded_model = tf.keras.models.load_model('/content/drive/MyDrive/CNN_IDS_mod
print("Model loaded successfully.")

# Load the scaler
loaded_scaler = joblib.load('/content/drive/MyDrive/scaler.save')
print("Scaler loaded successfully.")

# Load the label encoder
loaded_label_encoder = joblib.load('/content/drive/MyDrive/label_encoder_full
print("Label Encoder loaded successfully.")

# Re-initialize X and y, and prepare X_scaled (needed for sampling later if n
X = data.drop(columns=['Label'])
y = data['Label']

# Clean 'y' label if necessary (as done before)
y = y.str.replace("", "-", regex=False)

# Encode labels if not already encoded or if you need to recreate y_encoded
# This is important if you want to use y_encoded for further processing or if
label_encoder = loaded_label_encoder # Assign the loaded encoder to the name
y_encoded = label_encoder.fit_transform(y)

# Scale features (X_scaled is used for model input)
X_scaled = loaded_scaler.transform(X.values)

print("All necessary data and preprocessing tools have been reloaded.")
```

```
Main dataset loaded. Shape: (2827876, 79)
/usr/local/lib/python3.12/dist-packages/keras/src/saving/saving_lib.py:797: Us
saveable.load_own_variables(weights_store.get(inner_path))
Model loaded successfully.
Scaler loaded successfully.
Label Encoder loaded successfully.
All necessary data and preprocessing tools have been reloaded.
```

✓ Check the size of the **CICIDS2017.csv** file

```
import os

file_path = '/content/drive/MyDrive/CICIDS2017.csv'

if os.path.exists(file_path):
    file_size_bytes = os.path.getsize(file_path)
    file_size_mb = file_size_bytes / (1024 * 1024)
    print(f"The size of '{file_path}' is: {file_size_mb:.2f} MB")
else:
    print(f"File not found at '{file_path}'. Please ensure it was created cor
```

The size of '/content/drive/MyDrive/CICIDS2017.csv' is: 918.13 MB

## ✓ Data Analysis Key Findings

- A plot comparing training accuracy and validation accuracy over epochs was successfully generated to help identify overfitting or underfitting trends.
- A plot illustrating training loss against validation loss over epochs was successfully generated to provide insights into the model's learning progression.
- A heatmap of the confusion matrix was successfully generated, offering a visual overview of the model's classification performance by highlighting correctly and incorrectly classified instances for each class.
- A bar chart comparing the distribution of true and predicted labels on the test set was successfully generated, which is intended to help uncover potential biases in the model's predictions across different classes, particularly for minority classes.

Save the current `label_encoder` object (which was used to encode the labels for the model that achieved 99.72% accuracy) to Google Drive using `joblib`. This ensures that the attack labels can be consistently decoded when the best-performing model is loaded for future predictions.

```
# 98.75%
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatten, Dense, Dropout
from tensorflow.keras.optimizers import Adam

num_classes = len(label_encoder.classes_)

model = Sequential([
    Conv1D(filters=64, kernel_size=3, activation='relu',
           input_shape=(X_train_cnn.shape[1], 1)),
    MaxPooling1D(pool_size=2),
    Conv1D(filters=128, kernel_size=3, activation='relu'),
    MaxPooling1D(pool_size=2),

    Flatten(),
```

```
Dense(128, activation='relu'),
Dense(128, activation='relu'),
Dropout(0.1),

Dense(num_classes, activation='softmax')
])

model.compile(
    optimizer=Adam(learning_rate=0.01),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

[Show hidden output](#)

```
# 98.75%
from tensorflow.keras.callbacks import EarlyStopping, ReduceLRonPlateau

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=35,
    restore_best_weights=True
)

reduce_lr = ReduceLRonPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=10,
    min_lr=1e-5
)

history = model.fit(
    X_train_cnn,
    y_train,
    epochs=30,
    batch_size=256,
    validation_split=0.2,
    callbacks=[early_stop, reduce_lr],
    verbose=1
)
```

[Show hidden output](#)

```
test_loss, test_accuracy = model.evaluate(X_test_cnn, y_test, verbose=0)

print(f"Test Accuracy: {test_accuracy*100:.2f}%")
print(f"Test Loss: {test_loss:.4f}")
```

```
Test Accuracy: 98.75%  
Test Loss: 0.0339
```

```
# 98.75%  
from sklearn.metrics import classification_report  
import numpy as np  
  
y_pred = model.predict(X_test_cnn)  
y_pred_classes = np.argmax(y_pred, axis=1)  
  
print(classification_report(  
    y_test,  
    y_pred_classes,  
    target_names=label_encoder.classes_  
))
```

[Show hidden output](#)

```
# 98.75%  
plt.figure()  
plt.plot(history.history['accuracy'], label='Train Accuracy')  
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')  
plt.xlabel('Epochs')  
plt.ylabel('Accuracy')  
plt.title('Training vs Validation Accuracy')  
plt.legend()  
plt.show()
```

[Show hidden output](#)

## ✓ Load the saved model and preprocessing steps

```
# 98.75%  
import tensorflow as tf  
import joblib  
import numpy as np  
import pandas as pd  
  
# Load the saved CNN model  
loaded_model = tf.keras.models.load_model("/content/drive/MyDrive/CNN_IDS_mod  
print("Model loaded successfully.")  
  
# Load the scaler  
loaded_scaler = joblib.load("/content/drive/MyDrive/scaler.save")  
print("Scaler loaded successfully.")  
  
# Load the label encoder  
loaded_label_encoder = joblib.load("/content/drive/MyDrive/label_encoder_full  
print("Label Encoder loaded successfully.")
```



```
/usr/local/lib/python3.12/dist-packages/keras/src/saving/saving_lib.py:797: Us
saveable.load_own_variables(weights_store.get(inner_path))
Model loaded successfully.
Scaler loaded successfully.
Label Encoder loaded successfully.
```

```
# 98.75%
import joblib

joblib.dump(scaler, "/content/drive/MyDrive/scaler.save")
joblib.dump(label_encoder, "/content/drive/MyDrive/label_encoder.save")

print("Scaler and Label Encoder saved to Google Drive")
```

Scaler and Label Encoder saved to Google Drive

```
# Dropout and Learning Rate Changed
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatten, Dense, Dro
from tensorflow.keras.optimizers import Adam

num_classes = len(label_encoder.classes_)

model = Sequential([
    Conv1D(filters=64, kernel_size=3, activation='relu',
           input_shape=(X_train_cnn.shape[1], 1)),
    MaxPooling1D(pool_size=2),

    Conv1D(filters=128, kernel_size=3, activation='relu'),
    MaxPooling1D(pool_size=2),

    Flatten(),

    Dense(128, activation='relu'),
    Dropout(0.3), # Changed dropout rate from 0.1 to 0.3

    Dense(num_classes, activation='softmax')
])

model.compile(
    optimizer=Adam(learning_rate=0.001), # Learning rate changed from 0.1 to
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

[Show hidden output](#)

```
# Dropout and Learning Rate Changed
from tensorflow.keras.callbacks import EarlyStopping, ReduceLRonPlateau
```

```
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=35,
    restore_best_weights=True
)

reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=10,
    min_lr=1e-5
)

history = model.fit(
    X_train_cnn,
    y_train,
    epochs=30,
    batch_size=256,
    validation_split=0.2,
    callbacks=[early_stop, reduce_lr],
    verbose=1
)
```

[Show hidden output](#)

```
# Dropout and Learning Rate Changed
import matplotlib.pyplot as plt

# Plot training & validation accuracy values
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy with Dropout 0.3')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(loc='lower right')

# Plot training & validation loss values
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss with Dropout 0.3')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(loc='upper right')

plt.tight_layout()
plt.show()
```

[Show hidden output](#)

## Analysis of Training Graphs with Dropout 0.3

### Training vs Validation Accuracy Plot:

- **Observation:** Both training and validation accuracy show a consistent upward trend. The validation accuracy closely follows the training accuracy, and the gap between them is relatively small. Towards the end of training, both metrics appear to be converging or have converged at a high level.
- **Interpretation:** This indicates good model learning and generalization. The smaller gap between training and validation accuracy, compared to the previous run, suggests that the reduced dropout rate (0.3) has helped in mitigating overfitting. The model is learning effectively from the training data and performing well on unseen validation data.

### Training vs Validation Loss Plot:

- **Observation:** Both training and validation loss decrease steadily over epochs. The validation loss curve is smooth and does not show a significant increase or plateauing trend that would indicate strong overfitting.
- **Interpretation:** The synchronized decrease in both training and validation loss is a positive sign, indicating that the model is learning efficiently without memorizing the training data. The dropout rate of 0.3 seems to have improved the model's ability to generalize, as evidenced by the stable validation loss. This configuration appears to strike a better balance between bias and variance compared to the previous run with a dropout rate of 0.5.

## ✓ EXPERIMENTATION

### Analysis of Training History with Learning Rate 0.005

#### Training vs Validation Accuracy Plot (referring to the plot that would be generated):

- **Observation:** The training accuracy (`accuracy`) starts relatively lower but increases rapidly in the initial epochs. The validation accuracy (`val_accuracy`) also increases, but it shows significant fluctuations and starts to diverge from the training accuracy after a few epochs. It reaches a peak and then becomes more volatile. The model achieved a `val_accuracy` of around 97.7%-97.9% after dropping the learning rate, which is an improvement compared to the ~96% initial range but less stable than with a lower learning rate.
- **Interpretation:** The rapid increase in training accuracy indicates faster learning due to the higher learning rate. However, the larger gap and increased volatility between training and validation accuracy, especially after the initial epochs, are strong indicators of **overfitting**. The model is quickly memorizing the training

data, but this aggressive learning is causing it to perform less consistently on unseen validation data. The `ReduceLROnPlateau` callback effectively reduced the learning rate when `val_loss` plateaued, which helped to stabilize the accuracy somewhat towards the later epochs.

### Training vs Validation Loss Plot (referring to the plot that would be generated):

- **Observation:** The training loss (`loss`) decreases quickly. The validation loss (`val_loss`) also decreases initially but then starts to increase significantly and become erratic. It exhibits much more instability compared to the previous run with a lower learning rate. Even after the learning rate reduction by `ReduceLROnPlateau`, the `val_loss` remains higher and more erratic than ideal.
- **Interpretation:** The divergence and instability in the validation loss heavily suggest that the increased initial learning rate of 0.005 was too aggressive, leading to overfitting. While a higher learning rate can speed up convergence, in this case, it caused the model to overshoot optimal weights and struggle to generalize. The `EarlyStopping` callback likely prevented severe overfitting by restoring the best weights, but the overall training dynamics show that this learning rate was not ideal for stable generalization.

## Analysis of Training History

### Training vs Validation Accuracy Plot (from previous output):

- **Observation:** The training accuracy (`Train Accuracy`) shows a general upward trend, indicating that the model is learning from the training data. The validation accuracy (`Validation Accuracy`) also shows an upward trend and closely follows the training accuracy, but it appears to plateau or show minor fluctuations after a certain number of epochs.